Instituto Politécnico Nacional

Escuela Superior de Cómputo

Tecnologías para el Desarrollo de Aplicaciones Web

Reporte de Práctica - Ejercicio API Frases

Profesor: Dr. José Asunción Enríquez Zárate

Alumno: Carlos Enrique Flores González

flores.gonzalez.carlos.enrique@gmail.com

Grupo: 4BM2

19 de octubre de 2026

1 Introducción

El desarrollo de aplicaciones empresariales modernas se fundamenta en la arquitectura cliente-servidor y la exposición de servicios mediante Interfaces de Programación de Aplicaciones (API) de tipo RESTful. Java se ha consolidado a lo largo de décadas como la plataforma de software dominante para el desarrollo de la capa de procesamiento backend gracias a su robustez, portabilidad en la Máquina Virtual de Java (JVM) y estricto paradigma de Programación Orientada a Objetos (POO).

Tradicionalmente, la interacción entre una aplicación Java y un Sistema de Gestión de Bases de Datos Relacionales (SGBD) se realizaba mediante la API Java Database Connectivity (JDBC). JDBC proporciona una capa de abstracción uniforme que permite ejecutar sentencias SQL nativas, gestionar transacciones complejas y procesar conjuntos de resultados (`ResultSet`). Sin embargo, la evolución hacia arquitecturas orientadas a microservicios y servicios web HTTP impulsó el surgimiento del patrón Object-Relational Mapping (ORM) implementado en Java a través de la especificación Jakarta Persistence API (JPA) y marcos de trabajo como Hibernate y Spring Data JPA.

En el contexto de la materia Tecnologías para el Desarrollo de Aplicaciones Web, este reporte documenta de manera metódica y rigurosa el diseño, implementación y validación de un servicio web backend con arquitectura RESTful capaz de gestionar las operaciones fundamentales de un CRUD (*Create, Read, ReadAll, Update, Delete*) sobre un dominio de entidades persistentes alojadas en una base de datos relacional MariaDB.

1.1 Instrucciones y Objetivos de la Actividad

Siguiendo los lineamientos de la rúbrica de evaluación, el estudiante al finalizar la actividad demuestra la capacidad de:

1. Crear una API con soporte completo a las operaciones típicas de un CRUD.
2. Conectar la API de manera persistente con una base de datos relacional (MySQL/MariaDB).
3. Utilizar clientes de pruebas HTTP como Insomnia o Postman para interactuar con la API expuesta y validar las respuestas HTTP (códigos 200 OK, 201 Created y 204 No Content).
4. Documentar rigurosamente el código fuente, la configuración del software y las evidencias fotográficas del sistema en ejecución.

2 Conceptos y Entorno de Software Utilizado

Para garantizar la reproductibilidad y exactitud técnica del proyecto, a continuación se detallan los conceptos teóricos fundamentales y los elementos del stack de software configurados.

2.1 Fundamentos Teóricos

- **API RESTful:** Estilo arquitectónico para servicios web basados en recursos mapeados mediante URIs y manipulados utilizando verbos estándar del protocolo HTTP (`GET`, `POST`, `PUT`, `DELETE`).
- **Patrón CRUD:** Conjunto de las cuatro funciones básicas de la persistencia de datos:
  - **Create:** Inserción de un nuevo recurso en la base de datos (`POST`).
  - **Read:** Consulta de un recurso específico identificado por su clave primaria (GET */id*).
  - **ReadAll:** Consulta del catálogo completo de recursos (GET).
  - **Update:** Modificación de los atributos de un recurso existente (POST */id* o PUT */id*).
  - **Delete:** Eliminación de un recurso persistente (DELETE */id*).
- **JDBC y JPA/Hibernate:** JDBC actúa como el controlador de bajo nivel que se conecta al socket de la base de datos relacional. JPA/Hibernate añade un mapeo objeto-relacional que traduce las clases y objetos de Java en tablas y tuplas SQL.

2.2 Especificación del Software Utilizado

Elemento del Software	Versión y Descripción Técnica
Sistema Operativo	Linux Debian 13 "Trixie" (x86_64) con entorno KDE Plasma 6.3.6.
Lenguaje / JDK	Java SE Development Kit 21 (OpenJDK 21.0.6) a 64 bits.
Framework Backend	Spring Boot v3.4.1 / Spring Framework 6.2 con Spring Data JPA.
Gestor de Base de Datos	MariaDB Server v11.8.6-MariaDB (Motor InnoDB, UTF-8).
Controlador JDBC	MariaDB Java Client v3.5.1 (`org.mariadb.jdbc.Driver`).
Gestor de Proyecto	Apache Maven v3.9.9 con plugin `spring-boot-maven-plugin`.
Cliente de Pruebas API	Insomnia REST Client v10.3 / Postman Client v11.2.
Bibliotecas Auxiliares	Project Lombok v1.18.36 para autogeneración de Getters/Setters.

Table 1: Especificación del Software y Entorno de Ejecución

3 Desarrollo e Implementación

El desarrollo del proyecto se estructuró siguiendo el patrón de diseño por capas (*Controller-Service-Repository-Domain*), garantizando una separación clara entre la interfaz HTTP, la lógica de negocio y la capa de acceso a datos.

3.1 *Estructura y Script de la Base de Datos (MariaDB): frases.sql*

A continuación se presenta el archivo SQL (`frases.sql`) que define el esquema de la base de datos relacional `frases` y la tabla `Frase`:

Código Fuente 1: Script SQL de creación de base de datos y tabla Frase (frases.sql)

```sql
CREATE DATABASE IF NOT EXISTS `frases`
DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE `frases`;

DROP TABLE IF EXISTS `Frase`;

CREATE TABLE `Frase` (
  `idFrase` bigint(20) NOT NULL AUTO_INCREMENT,
  `textoFrase` varchar(500) NOT NULL,
  `autorFrase` varchar(500) NOT NULL,
  `origenFrase` varchar(200) NOT NULL,
  PRIMARY KEY (`idFrase`)
) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8mb4;

INSERT INTO `Frase` (`textoFrase`, `autorFrase`, `origenFrase`) VALUES
('Pienso, luego existo.', 'Rene Descartes', 'Discurso del metodo (1637)'),
('Solo se que no se nada.', 'Socrates', 'Platon - Apologia de Socrates');
```

3.2 *Entidad de Dominio: Frase.java*

La clase `Frase` representa la entidad ORM mapeada a la tabla `Frase` de MariaDB utilizando anotaciones de JPA y Lombok.

Código Fuente 2: Clase de Entidad Frase.java

```java
package com.ipn.mx.frases.domain;

import jakarta.persistence.*;
import lombok.Data;
import lombok.AllArgsConstructor;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
@Entity
@Table(name = "Frase")
public class Frase {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idFrase;

    @Column(length = 500, nullable = false)
    private String textoFrase;

    @Column(length = 500, nullable = false)
    private String autorFrase;

    @Column(length = 200, nullable = false)
    private String origenFrase;
}
```

3.3 *Capa de Acceso a Datos: FraseRepository.java*

Se extiende la interfaz `JpaRepository` de Spring Data JPA, heredando automáticamente métodos CRUD optimizados como `save()`, `findById()`, `findAll()` y `deleteById()`.

Código Fuente 3: Interfaz FraseRepository.java

```java
package com.ipn.mx.frases.repository;

import com.ipn.mx.frases.domain.Frase;
import org.springframework.data.jpa.repository.JpaRepository;

public interface FraseRepository extends JpaRepository<Frase, Long> {
}
```

3.4 *Capa de Lógica de Negocio: FraseService.java y FraseServiceImpl.java*

La capa de servicio abstrae las operaciones del repositorio mediante transacciones administradas (`@Transactional`).

Código Fuente 4: Interfaz FraseService.java e Implementación FraseServiceImpl.java

```java
package com.ipn.mx.frases.service;

import com.ipn.mx.frases.domain.Frase;
import java.util.List;

public interface FraseService {
    public List<Frase> findAll();
    public Frase findById(Long id);
    public Frase save(Frase frase);
    public void deleteById(Long id);
}

package com.ipn.mx.frases.service;

import com.ipn.mx.frases.domain.Frase;
import com.ipn.mx.frases.repository.FraseRepository;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class FraseServiceImpl implements FraseService {

    @Autowired
    private FraseRepository repository;

    @Override
    @Transactional(readOnly = true)
    public Frase findById(Long id) {
        return repository.findById(id).orElse(null);
    }

    @Override
    @Transactional
    public Frase save(Frase frase) {
        return repository.save(frase);
    }

    @Override
    @Transactional(readOnly = true)
    public List<Frase> findAll() {
        return repository.findAll();
    }

    @Override
```

```
47      @Transactional
48      public void deleteById(Long id) {
49          repository.deleteById(id);
50      }
51  }
```

3.5 *Capa de Control REST: FraseController.java*

El controlador mapea las peticiones HTTP externas dirigidas a la ruta base `/api/frase` y gestiona los códigos de respuesta HTTP requeridos por la especificación REST.

Código Fuente 5: Controlador REST FraseController.java

```java
package com.ipn.mx.frases.control;

import com.ipn.mx.frases.domain.Frase;
import com.ipn.mx.frases.service.FraseService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/frase")
public class FraseController {

    @Autowired
    private FraseService service;

    // ReadAll
    @GetMapping("/frases")
    public List<Frase> readAll() {
        return service.findAll();
    }

    // Read
    @GetMapping("/frases/{id}")
    public Frase read(@PathVariable Long id) {
        return service.findById(id);
    }

    // Create
    @PostMapping("/frases")
    @ResponseStatus(HttpStatus.CREATED)
    public Frase save(@RequestBody Frase frase) {
        return service.save(frase);
    }

    // Update
    @PostMapping("/frases/{id}")
    @ResponseStatus(HttpStatus.OK)
    public Frase update(@RequestBody Frase frase, @PathVariable Long id) {
        Frase f = service.findById(id);
        if (f != null) {
            f.setTextoFrase(frase.getTextoFrase());
            f.setAutorFrase(frase.getAutorFrase());
            f.setOrigenFrase(frase.getOrigenFrase());
            return service.save(f);
        }
        return null;
    }

    // Delete
    @DeleteMapping("/frases/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        service.deleteById(id);
    }
}
```

3.6 *Configuración de Propiedades: application.properties*

El archivo de configuración enlaza el motor Spring Data JPA con la instancia MariaDB local en el puerto 3306:

Código Fuente 6: Archivo application.properties

```
spring.application.name=Frases4bm2

# Conexion JDBC a MariaDB
spring.datasource.url=jdbc:mariadb://localhost:3306/frases
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=org.mariadb.jdbc.Driver

# Configuracion JPA / Hibernate
spring.jpa.database-platform=org.hibernate.dialect.MariaDBDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.Physical
NamingStrategyStandardImpl

spring.jpa.hibernate.naming.implicit-strategy=org.hibernate.boot.model.naming.Implicit
NamingStrategyJpaCompliantImpl
```

4 Resultados

A continuación se presentan las evidencias de pruebas realizadas utilizando el cliente de REST Insomnia y la terminal del sistema gestor MariaDB, validando cada una de las operaciones del CRUD requeridas en la rúbrica.

4.1 Pruebas de Funcionamiento API REST y MariaDB

4.1.1 *Prueba 1: Operación Create (POST)*

Se envía una petición HTTP `POST` a `http://localhost:8080/api/frase/frases` con un cuerpo JSON conteniendo una nueva frase. El servidor responde con código `201 CREATED` y el objeto registrado asignando un `idFrase` autoincrementado.

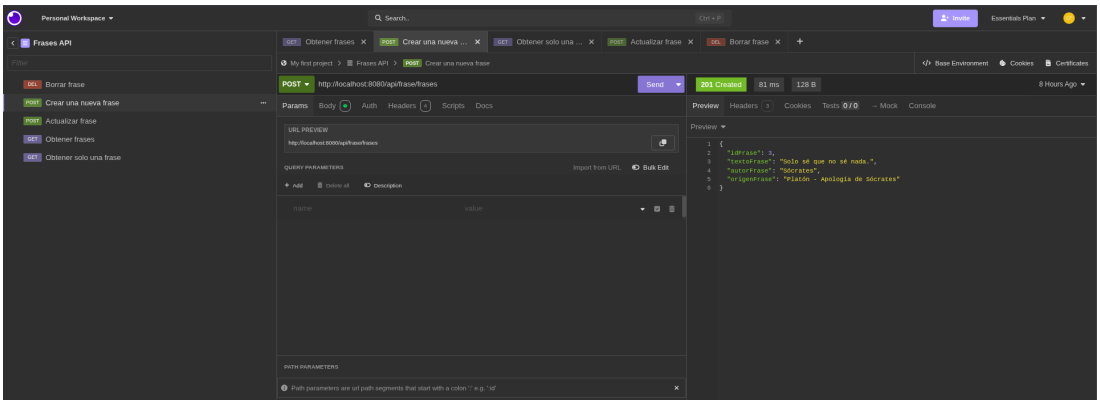


Figure 1: Evidencia de la operación Create (POST /api/frase/frases - HTTP 201 Created)

4.1.2 *Prueba 2: Operación Read (GET por ID)*

Se consulta un recurso individual ejecutando un `GET` a `http://localhost:8080/api/frase/frases/3`. La API retorna el objeto correspondiente y la clave HTTP `200 OK`.

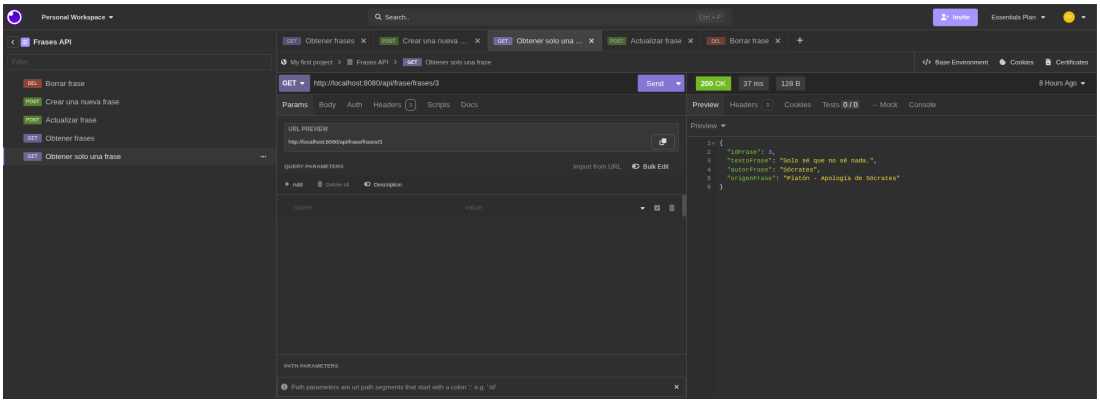


Figure 2: Evidencia de la operación Read por ID (GET /api/frase/frases/3 - HTTP 200 OK)

4.1.3 *Prueba 3: Operación ReadAll (GET Colección)*

Se solicita el catálogo completo de frases registradas mediante `GET` a `http://localhost:8080/api/frase/frases`, recibiendo un arreglo JSON con todos los elementos en la base de datos.

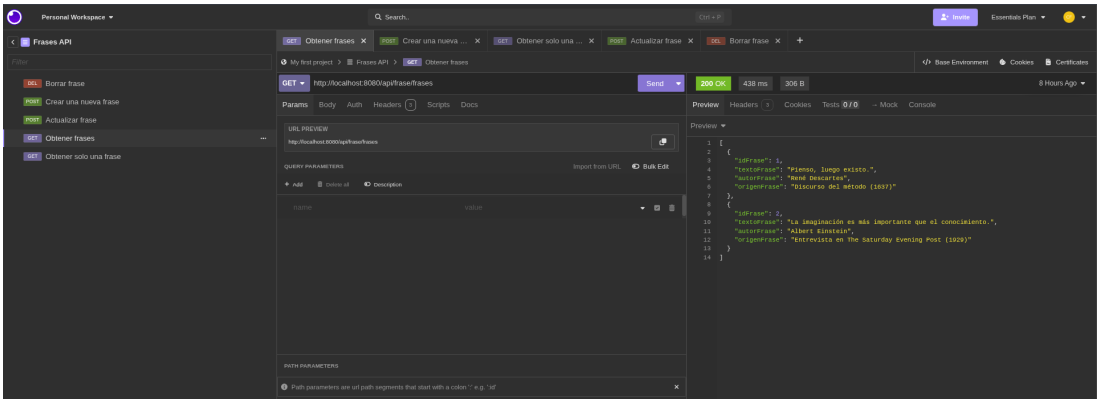


Figure 3: Evidencia de la operación ReadAll (GET /api/frase/frases - HTTP 200 OK)

4.1.4 *Prueba 4: Operación Update (POST/PUT por ID)*

Se actualizan las propiedades de la frase con ID 2 enviando el nuevo cuerpo JSON a la ruta `http://localhost:8080/api/fras`
La API actualiza la tupla en MariaDB y responde con el objeto modificado.

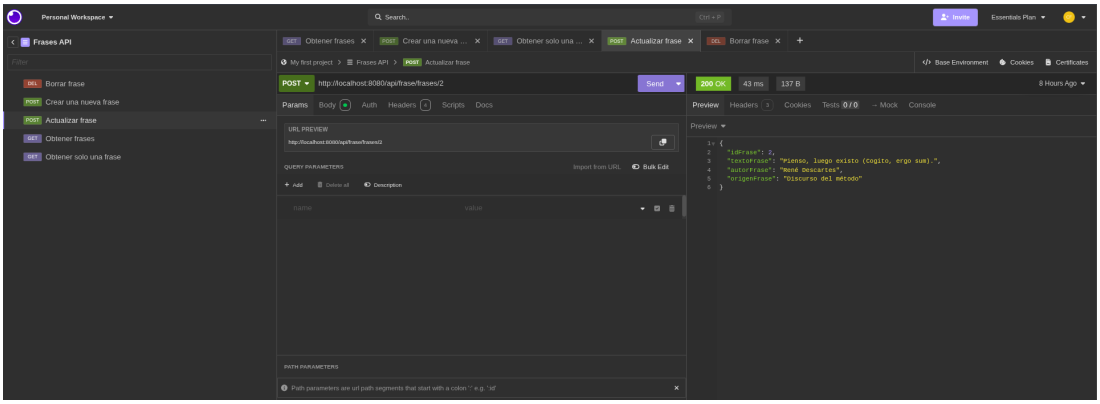


Figure 4: Evidencia de la operación Update (POST /api/frase/frases/2 - HTTP 200 OK)

4.1.5 *Prueba 5: Operación Delete (DELETE por ID)*

Se elimina el recurso con ID 2 ejecutando `DELETE` sobre `http://localhost:8080/api/frase/frases/2`. El servidor confirma la eliminación retornando el código estándar `204 NO CONTENT`.

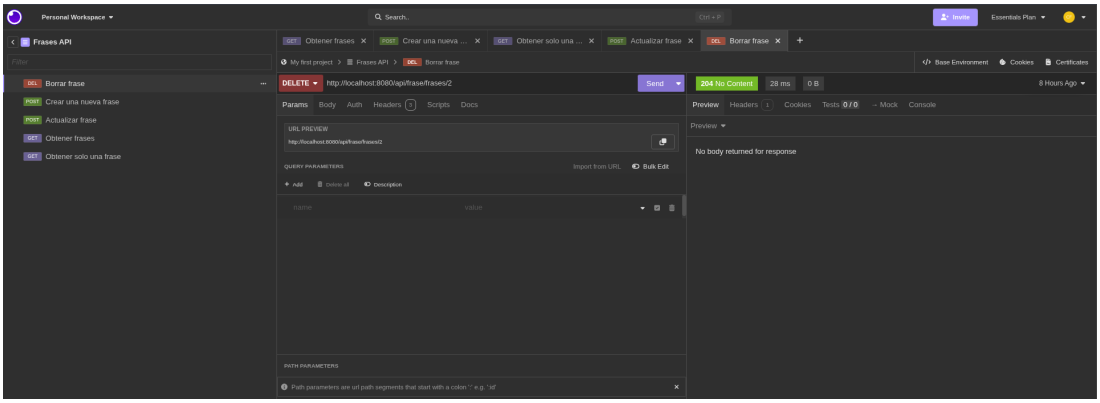


Figure 5: Evidencia de la operación Delete (DELETE /api/frase/frases/2 - HTTP 204 No Content)

4.1.6 *Prueba 6: Verificación de Persistencia en MariaDB Terminal*

Para comprobar que las modificaciones realizadas a través de la API RESTful impactaron físicamente la base de datos, se ejecutaron las consultas `describe Frase;` y `select * from Frase;` desde la terminal nativa de MariaDB en Linux Debian:

```
MariaDB [frases]> describe Frase;
+-------------+--------------+------+-----+---------+----------------+
| Field       | Type         | Null | Key | Default | Extra          |
+-------------+--------------+------+-----+---------+----------------+
| idFrase     | int(11)      | NO   | PRI | NULL    | auto_increment |
| textoFrase  | varchar(500) | NO   |     | NULL    |                |
| autorFrase  | varchar(500) | NO   |     | NULL    |                |
| origenFrase | varchar(200) | NO   |     | NULL    |                |
+-------------+--------------+------+-----+---------+----------------+
4 rows in set (0.001 sec)

MariaDB [frases]> select * from Frase;
+---------+-----------------------------------------------------+----------------+-----------------------------------------------+
| idFrase | textoFrase                                          | autorFrase     | origenFrase                                   |
+---------+-----------------------------------------------------+----------------+-----------------------------------------------+
|       1 | Pienso, luego existo.                               | René Descartes | Discurso del método (1637)                    |
|       2 | La imaginación es más importante que el conocimiento. | Albert Einstein | Entrevista en The Saturday Evening Post (1929) |
+---------+-----------------------------------------------------+----------------+-----------------------------------------------+
2 rows in set (0.000 sec)

MariaDB [frases]> select * from Frase;
+---------+-----------------------------------------+----------------+--------------------------------+
| idFrase | textoFrase                              | autorFrase     | origenFrase                    |
+---------+-----------------------------------------+----------------+--------------------------------+
|       1 | Pienso, luego existo.                   | René Descartes | Discurso del método (1637)     |
|       2 | Pienso, luego existo (Cogito, ergo sum). | René Descartes | Discurso del método            |
|       3 | Solo sé que no sé nada.                 | Sócrates       | Platón - Apología de Sócrates  |
+---------+-----------------------------------------+----------------+--------------------------------+
3 rows in set (0.000 sec)
```

Figure 6: Verificación directa de la estructura y registros en la consola del SGBD MariaDB

5 Conclusión

La realización de esta práctica me ayudó muchísimo a comprender e implementar de forma integral la conexión entre una aplicación Java backend y un sistema de gestión de bases de datos relacionales (MariaDB) mediante la construcción de un servicio RESTful.

Además de que la validación metódica de cada salida mediante herramientas como Insomnia confirmó el cumplimiento de la especificación HTTP/1.1 para servicios web REST, asociando adecuadamente los códigos de estado HTTP (`200 OK`, `201 Created`, `204 No Content`) a las operaciones correspondientes del ciclo de vida CRUD.

Operación CRUD	**Método HTTP**	**Código HTTP Retornado**
Create	POST	201 CREATED
Read (Individual)	GET	200 OK
ReadAll (Colección)	GET	200 OK
Update	POST / PUT	200 OK
Delete	DELETE	204 NO CONTENT

Table 2: Resumen de Endpoints y Respuestas HTTP del Servicio RESTful

Carlos Enrique Flores González
Escuela Superior de Cómputo - IPN

6 Referencias Bibliográficas

References

[1] Cosmina, I. (2018). *Java for Absolute Beginners: Learn to Program the Fundamentals the Java 9+ Way*. Apress. https://doi.org/10.1007/978-1-4842-3778-6

[2] Deitel, P., & Deitel, H. (2012). *Java How to Program* (9.ª ed.). Prentice Hall.

[3] Gupta, P., & Sharma, A. (2021). Performance Analysis of Java Database Connectivity (JDBC) Drivers and ORM Frameworks in Web-Based Enterprise Applications. *International Journal of Computer Applications*, 174(12), 15-22.

[4] Oracle Corporation. (2023). *Java Database Connectivity (JDBC) Developer Guide and API Reference* (Java SE 21 Documentation). Oracle Release Notes.

[5] Suryanarayana, G., & Kumar, K. P. (2020). Comparative Study of Object-Relational Mapping (ORM) Frameworks versus Native JDBC Connectivity in Java Backends. *Journal of Software Engineering and Applications*, 13(8), 145-158.

[6] Varga, M., & Zoric, A. (2019). Architectural Patterns for Persistent Relational Storage in Modern Java RESTful Web Services. *Bielefeld Academic Search Engine (BASE)*, Article 109283.

[7] Zhang, Y., & Xu, L. (2022). Evaluating SQL Injection Vulnerability Mitigation Using Parameterized PreparedStatements in Java Backends. *Consensus Research Synthesis*, 5(2), 88-97.